

Guía de Aprendizaje: Punteros en C

Monitorías de Sistemas Operativos

1. Motivación

En muchos programas necesitamos trabajar directamente con direcciones de memoria. Esto permite modificar variables desde funciones, manejar estructuras dinámicas o interactuar con dispositivos y sistemas operativos. Para ello, C utiliza **punteros**.

2. Idea Fundamental

Un puntero es una **variable** que almacena la **dirección de memoria** de otra variable.

3. Sintaxis Básica

La declaración de un puntero sigue el formato:

```
tipo_de_dato *nombre_del_puntero;
```

4. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2
3 int main(){
4     int x = 10;
5     int *p = &x; // p almacena la direccion de x
6
7     printf("Valor de x: %d\n", x);
8     printf("Valor a traves del puntero: %d\n", *p);
9
10    return 0;
11 }
```

5. Explicación Paso a Paso

1. Se declara una variable **x** con el valor 10.
2. Se declara un puntero **p**.
3. Con el operador **&** (referencia), **p** almacena la dirección de memoria de **x**.
4. Con el operador ***** (dereferencia), ***p** accede al valor almacenado en esa dirección.

6. Qué ocurre en memoria

Conceptualmente, la memoria se organiza de la siguiente manera:

Variable	Dirección	Valor
x	0x100	10
p	0x200	0x100

Explicación: p almacena la dirección de x. Al usar *p, el procesador va a la dirección 0x100 y lee el valor allí almacenado.

7. Patrones comunes de uso

El uso más común de los punteros es el **Paso por Referencia** para modificar variables dentro de funciones:

```
1 void duplicar(int *n){  
2     *n = (*n) * 2;  
3 }
```

8. Errores Comunes

- **Puntero no inicializado:** Intentar usar un puntero que no apunta a nada válido (Segmentation Fault).
- **Confusión entre * y &:** Usar el operador incorrecto al asignar o acceder.

9. Buenas Prácticas

- Inicializar siempre los punteros (usar NULL si no tienen destino aún).
- Usar nombres descriptivos (ej: ptr_edad).

10. Ejercicios

1. Escriba una función que intercambie el valor de dos enteros usando punteros.
2. Cree un programa que imprima la dirección de memoria de una variable y luego la dirección de memoria del puntero que la apunta.